Football Career Legacy

Documentação técnica e evidências de implantação

Kauã do Sacramento Soares · FIAP · Cibersegurança
02 de novembro de 2026

O Football Career Legacy é uma aplicação web de modo carreira de futebol. O usuário pode atuar como técnico de um clube ou desenvolver um jogador, disputando temporadas, acompanhando resultados e mantendo o progresso em SQLite.

Este relatório apresenta a arquitetura, as funções implementadas e as evidências de implantação na Microsoft Azure. A aplicação usa FastAPI e identifica sua API como versão 2.1.0, com frontend em HTML, CSS e JavaScript. O conjunto de imagens registra instalação, testes, backup e uso da interface.

Identificação	Valor
Repositório e branch	github.com/kauasoares/football-career-api · master
Aplicação registrada nos prints	http://20.104.57.183/
Swagger	http://20.104.57.183/docs
Banco	SQLite · data/career.db
Formato da entrega	Versão resumida com 18 figuras selecionadas

As evidências foram selecionadas entre os prints recebidos para evitar repetições. As telas de frontend representam a versão anterior ao refinamento visual final; o texto descreve também as melhorias já incorporadas ao código. O endereço público é o registrado nas capturas, sem uma nova verificação de disponibilidade.

Leitura	Páginas
Arquitetura e infraestrutura	2
Dados e contratos da API	3
Modos de jogo	4
Instalação e atualização	5
Backup e validação	6
Evidências selecionadas	7 a 15

Arquitetura e infraestrutura

O navegador acessa o IP público da VM. O NSG filtra as conexões e o Nginx recebe HTTP na porta 80. Os arquivos do frontend são servidos pelo Nginx; as rotas da API são encaminhadas ao Uvicorn em 127.0.0.1:8000. O FastAPI executa as regras e consulta o SQLite. O systemd mantém o backend ativo.

Recurso	Configuração registrada
Assinatura	Azure for Students
Grupo de recursos e região	rg-football-career · Canada Central
Máquina virtual	vm-football-career
Imagem e tamanho selecionados	Ubuntu Server 24.04 LTS x64 Gen2; Standard_B2ats_v2; 2 vCPUs e 1 GiB
Disco selecionado	SSD Standard; imagem padrão de 30 GiB
VNet e sub-rede	vnet-canadacentral-1 / snet-canadacentral-1
Rede e IP privado	172.16.0.0/24 · 172.16.0.4
IP público	20.104.57.183
Grupo de segurança	vm-football-career-nsg
Diretório e serviço	/opt/football-career-api · football-career-api.service
Versões no terminal	Python 3.12.3; FastAPI 0.115.6; Uvicorn 0.32.1; Nginx 1.24.0; SQLite 3.45.1

As telas de criação comprovam os parâmetros selecionados. Os prints posteriores mostram o serviço ativo, o Swagger público e a interface acessível. O ambiente acadêmico utiliza HTTP e não implementa contas ou autenticação.

Na captura do NSG, TCP 22 e 80 estão liberados a partir de qualquer origem. Restringir SSH ao IP administrativo é uma melhoria recomendada, não uma configuração comprovada pelas imagens. A porta 8000 permanece vinculada ao endereço local no comando do Uvicorn.

Dados e contratos da API

Cada carreira cria seu próprio mundo com oito clubes, elencos e calendário. As sete tabelas são careers, teams, players, fixtures, goals, season_stats e offers. O campo career_id separa os mundos; PRAGMA foreign_keys = ON e restrições SQL reforçam a integridade. Esse isolamento não substitui autorização de usuários.

Grupo	Rotas principais
Sistema e carreiras	GET /health; GET /clubs; GET e POST /careers; GET e DELETE /careers/{cid}
Liga	GET /careers/{cid}/standings, /fixtures, /teams, /teams/{team_id}, /squad e /top-scorers
Progressão	POST /careers/{cid}/advance e /next-season
Técnico	PUT /careers/{cid}/lineup; GET /market; POST /market/buy e /market/sell
Jogador	GET /careers/{cid}/player e /offers; POST /offers/{offer_id}/accept e /retire

Nas rotas abreviadas da tabela, /market, /player, /offers e /retire seguem o prefixo /careers/{cid}. O Swagger em /docs contém o catálogo completo, os parâmetros e as validações. O esquema também está em /openapi.json.

```
POST /careers
{"name":"Minha carreira","mode":"MANAGER",
"club":"Tigres do ABC"}

POST /careers
{"name":"Minha carreira como jogador","mode":"PLAYER",
"player":{"name":"Kaua Soares","position":"ATT","age":18}}
```

O clube deve corresponder ao catálogo, incluindo acentos. Posições válidas: GK, DEF, MID e ATT; ATA é rejeitada. O jogador inicia entre 16 e 21 anos. A API retorna mensagens em detail para entradas inválidas.

O perfil retorna atributos, history por temporada e career_totals de jogos, gols e assistências. Propostas retornam transfer_window e offers; cada oferta informa clube, papel, força, última posição e status. Uma lista vazia com janela fechada é um retorno válido.

Modos de jogo e uso da interface

A temporada tem 14 rodadas, com turno e returno entre oito clubes. A simulação considera titulares, posições, ratings, tática e mando de campo. Após a última rodada, Iniciar próxima temporada realiza avaliação, evolução dos atletas e criação do novo calendário.

Modo técnico	Modo jogador
Escolha do clube, orçamento e confiança da diretoria	Criação do atleta, idade, posição, overall e potencial
Campo e cards por posição, titulares e reservas	Perfil dedicado, valor, estatísticas e histórico
Formações 4-4-2, 4-3-3 e 3-5-2	Propostas de transferência e confirmação de aceitação
Seleção manual ou automática seguida de salvamento	Atualização de perfil, clube, tabela e calendário após transferir

Escalação: selecionar formação e tática, retirar um titular da posição desejada e escolher o reserva. Conferir os limites por posição e salvar. São exigidos 11 IDs distintos do próprio elenco: 1 goleiro e a distribuição de defensores, meias e atacantes da formação. A seleção automática prepara os titulares; também exige salvar para aplicar.

A confiança do técnico começa em 60, aumenta 4 após vitória e cai 4 após derrota. A avaliação final compara a posição com a meta. Confiança de 15 ou menos encerra a carreira. O jogador evolui conforme idade e desempenho e se aposenta automaticamente aos 36 anos.

Transferências do jogador: a janela abre em carreira ativa antes das rodadas 1 e 8. Até três ofertas podem ser geradas na mudança de temporada, conforme o rating do atleta e o elenco de outros clubes. Janela aberta não garante ofertas. Aceitar exige oferta pendente e confirmação; a API muda o clube e recusa as demais propostas pendentes.

Classificação e calendário são compartilhados pelos dois modos. Os totais do atleta incluem a temporada atual; sem partidas, a nota aparece como não disponível. Compra e venda do técnico, artilharia, exclusão e aposentadoria manual permanecem acessíveis pela API, sem tela dedicada.

Instalação e atualização

A implantação fotografada usa Ubuntu 24.04, usuário azureuser e /opt/football-career-api. Após instalar Python, Git e Nginx, o projeto é clonado e executado em ambiente virtual. Estes comandos são referência de reprodução, não ações realizadas por este relatório.

```
git clone --branch master \
https://github.com/kauasoares/football-career-api.git \
/opt/football-career-api
cd /opt/football-career-api
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

O serviço football-career-api.service deve iniciar Uvicorn com main:app, host 127.0.0.1, porta 8000 e WorkingDirectory no repositório. DB_PATH indica o arquivo SQLite. Os prints mostram o serviço active (running) e a API local respondendo.

```
sudo systemctl enable --now football-career-api
systemctl status football-career-api --no-pager
curl http://127.0.0.1:8000/health
```

O Nginx deve servir /opt/football-career-api/frontend e encaminhar as rotas da API ao Uvicorn. O frontend usa const API = "", portanto HTML e API precisam estar na mesma origem HTTP. O arquivo completo de configuração de referência está no README do repositório.

```
sudo nginx -t
sudo systemctl reload nginx
curl http://127.0.0.1/health
```

Para atualizar a interface na instalação existente:

```
cd /opt/football-career-api
git pull origin master
```

Recarregar o navegador com Ctrl + F5. Quando o Nginx já serve a pasta frontend diretamente, mudanças somente nos arquivos estáticos dispensam reinício do backend. O erro version is not defined foi corrigido em dde372b ao inicializar o controle de carregamento do dashboard.

Backup validação e continuidade

backup_db.py usa o backup online do SQLite e valida a cópia com PRAGMA integrity_check. DB_PATH seleciona a origem, BACKUP_DIR define o destino e BACKUP_RETENTION_DAYS controla a retenção, com padrão de 14 dias.

```
cd /opt/football-career-api
source .venv/bin/activate
python backup_db.py
```

As imagens comprovam a criação do backup career-20261102T212136Z.db e a abertura de uma cópia em /tmp/football-career-restore. A integridade retornou ok e a consulta encontrou as carreiras de técnico e jogador. Isso valida a cópia consultada, sem comprovar substituição do banco ativo ou agendamento por cron.

Validação registrada	Resultado e limite
Servidor e proxy	Uvicorn local, systemd ativo e Nginx respondendo /health
Acesso externo	Swagger e frontend exibidos no IP público
Persistência	Banco com tabelas e carreira consultada após criação via API
Entradas inválidas	Clube inválido retorna 400; posição ATA retorna 422
Modos de jogo	Criação, simulação, tabela, calendário e formação fotografados
Propostas	Janela fechada e lista vazia; sem captura de aceitação real
Testes automatizados	Nove casos no repositório; não há print de pytest na Azure

O diagnóstico inicial registrou falha SSH por permissões da chave e timeout do acesso público. As imagens posteriores mostram login e Swagger funcionando, sem permitir atribuir a causa de rede a um único componente. O erro do frontend foi corrigido no código e o usuário confirmou o funcionamento posterior.

Limitações: ausência de autenticação e HTTPS, SQLite vinculado ao disco da VM e algumas funções apenas pela API. Evoluções possíveis: controle de acesso, tela do mercado do técnico, monitoramento, cópia externa de backups e testes de navegador. Nenhuma mudança no banco ou deploy na VM foi realizada para produzir este relatório.

Fontes do projeto: README.md, main.py, backup_db.py, tests/test_api.py e frontend/. As capturas do anexo sustentam somente os estados exibidos; a interface atual inclui melhorias posteriores ao layout fotografado.

Evidências de provisionamento

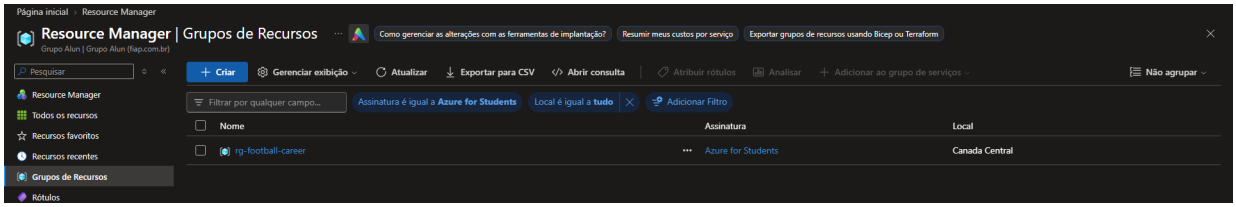


Figura 1 — Grupo de recursos rg football career

O recurso aparece na região Canada Central.

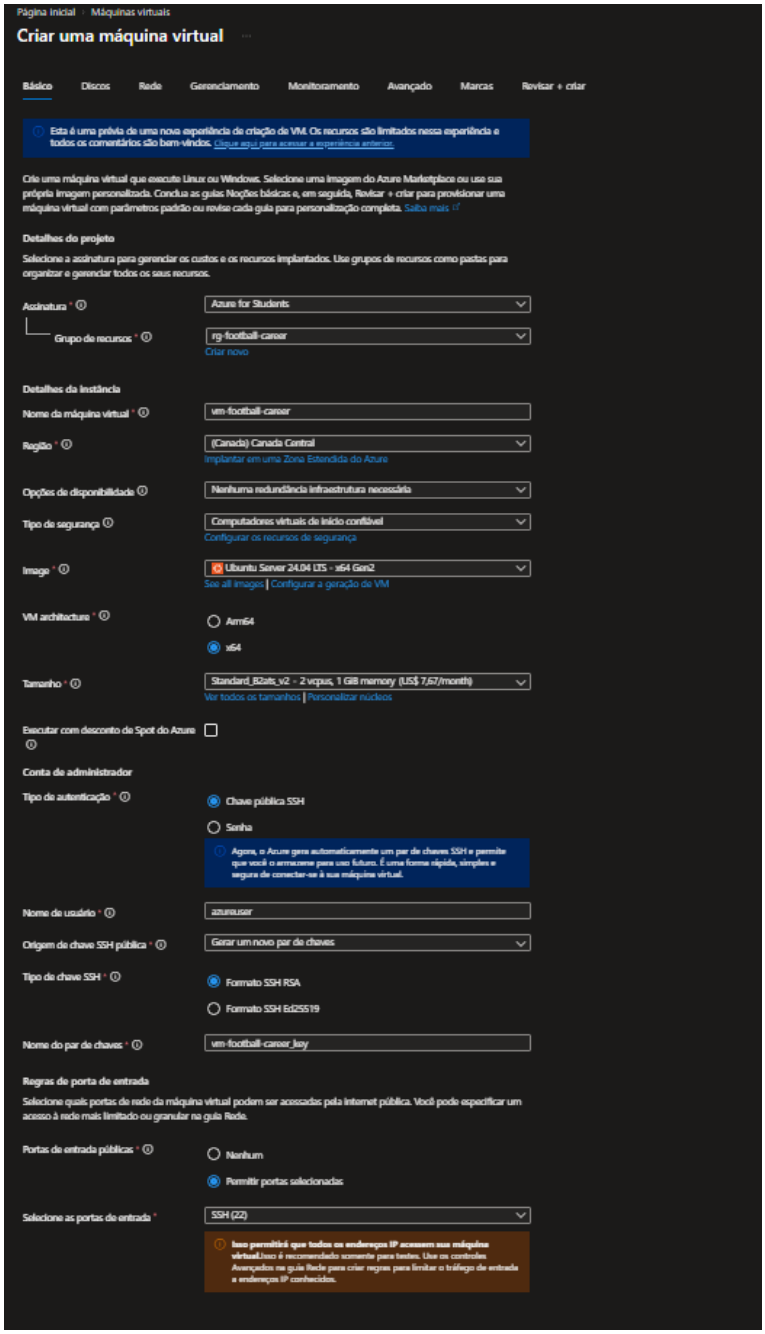


Figura 2 — Parâmetros básicos de criação da VM

Imagem Ubuntu Server 24.04 LTS x64, Standard_B2ats_v2 e autenticação por chave SSH selecionados.

Evidências de rede e serviço

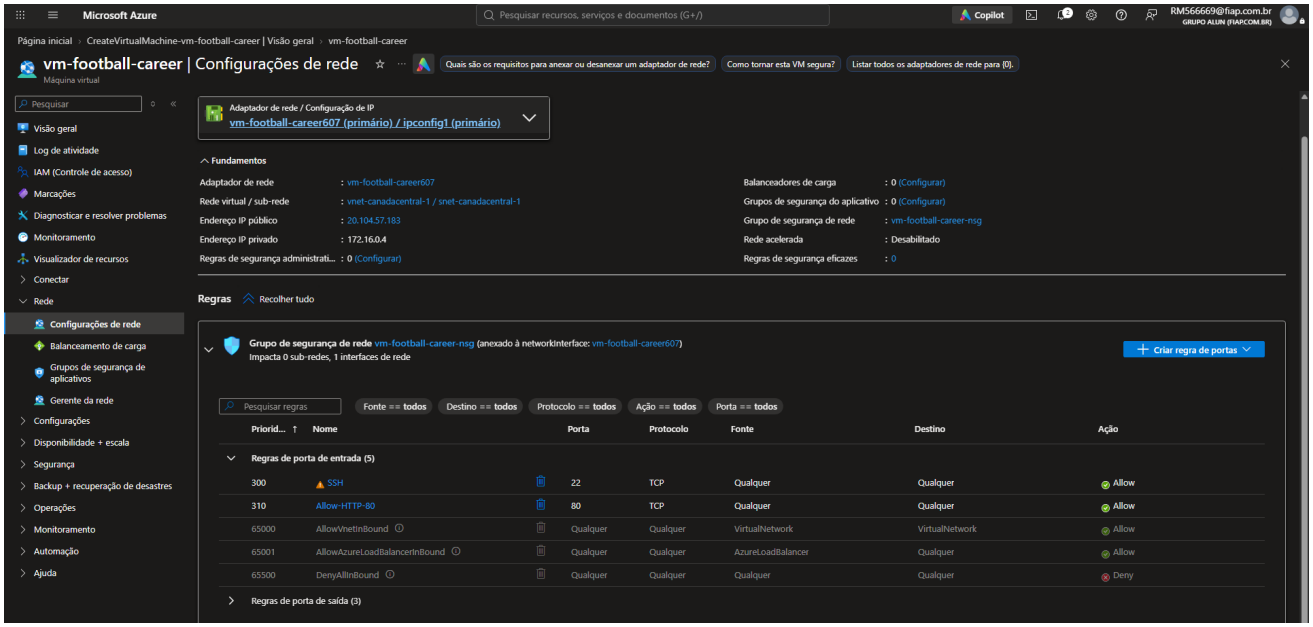


Figura 3 — Configuração do NSG

Regras de entrada permitem SSH 22 e HTTP 80; a origem SSH exibida é qualquer origem.

```
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sudo systemctl daemon-reload
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sudo systemctl enable --now football-career-api
Created symlink /etc/systemd/system/multi-user.target.wants/football-career-api.service → /etc/systemd/system/footbal
l-career-api.service.
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sudo systemctl status football-career-api --no-pager
● football-career-api.service - Football Career API - FastAPI
     Loaded: loaded (/etc/systemd/system/football-career-api.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-11-02 20:29:34 UTC; 7s ago
   Main PID: 4540 (uvicorn)
      Tasks: 5 (limit: 472)
     Memory: 29.5M (peak: 29.7M)
        CPU: 372ms
     CGroup: /system.slice/football-career-api.service
             └─4540 /opt/football-career-api/.venv/bin/python /opt/football-career-api/.venv/bin/uvicorn main:app --…

Nov 02 20:29:34 vm-football-career systemd[1]: Started football-career-api.service - Football Career API - FastAPI.
Nov 02 20:29:34 vm-football-career uvicorn[4540]: INFO:     Started server process [4540]
Nov 02 20:29:34 vm-football-career uvicorn[4540]: INFO:     Waiting for application startup.
Nov 02 20:29:34 vm-football-career uvicorn[4540]: INFO:     Application startup complete.
Nov 02 20:29:34 vm-football-career uvicorn[4540]: INFO:     Uvicorn running on http://127.0.0.1:8000 (Press C…o quit)
Hint: Some lines were ellipsized, use -l to show in full.
(.venv) azureuser@vm-football-career:/opt/football-career-api$
```

Figura 4 — Serviço systemd habilitado

football-career-api.service aparece active (running).

```
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Figura 5 — Validação do Nginx

nginx -t informa sintaxe válida e teste bem-sucedido.

Evidências de acesso à API

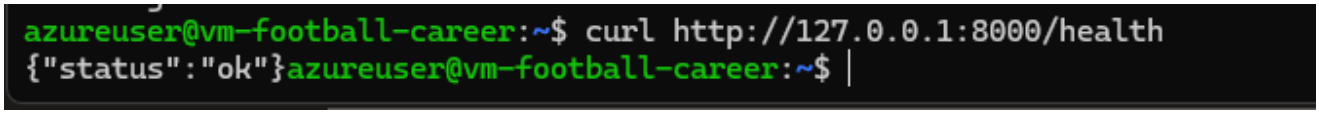


Figura 6 — Saúde da API local

curl retorna status ok na porta 8000.

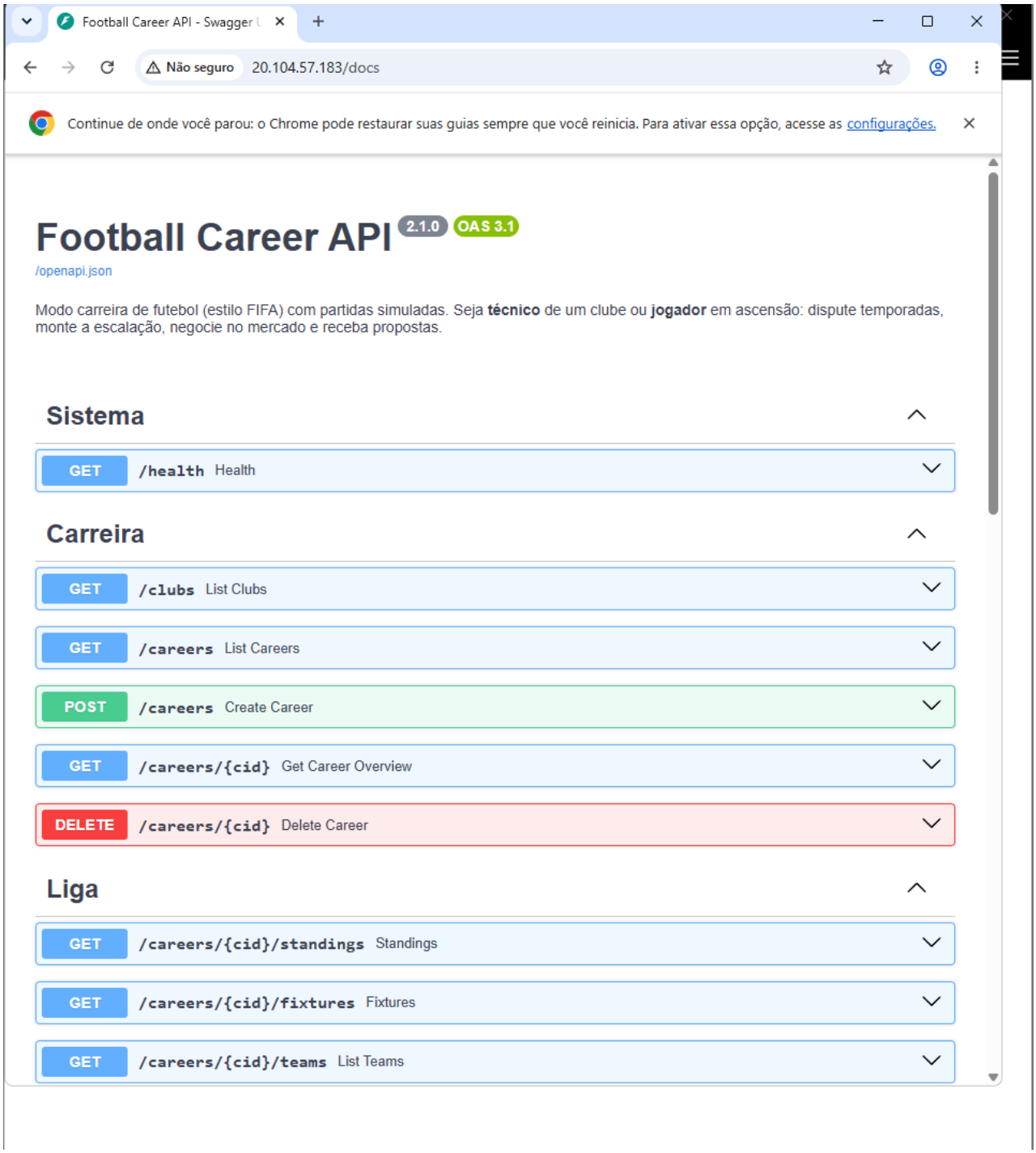


Figura 7 — Swagger pelo IP público

A documentação Football Career API 2.1.0 carrega em 20.104.57.183/docs.

Evidências de criação e persistência

```
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sqlite3 data/career.db "SELECT * FROM careers;"
1|Carreira Teste Azure|MANAGER|ACTIVE|1|1|3||60|3|2026-11-02 20:46:27
(.venv) azureuser@vm-football-career:/opt/football-career-api$
```

Figura 8 — Registro persistido no banco

Consulta SELECT em careers mostra a carreira MANAGER criada.

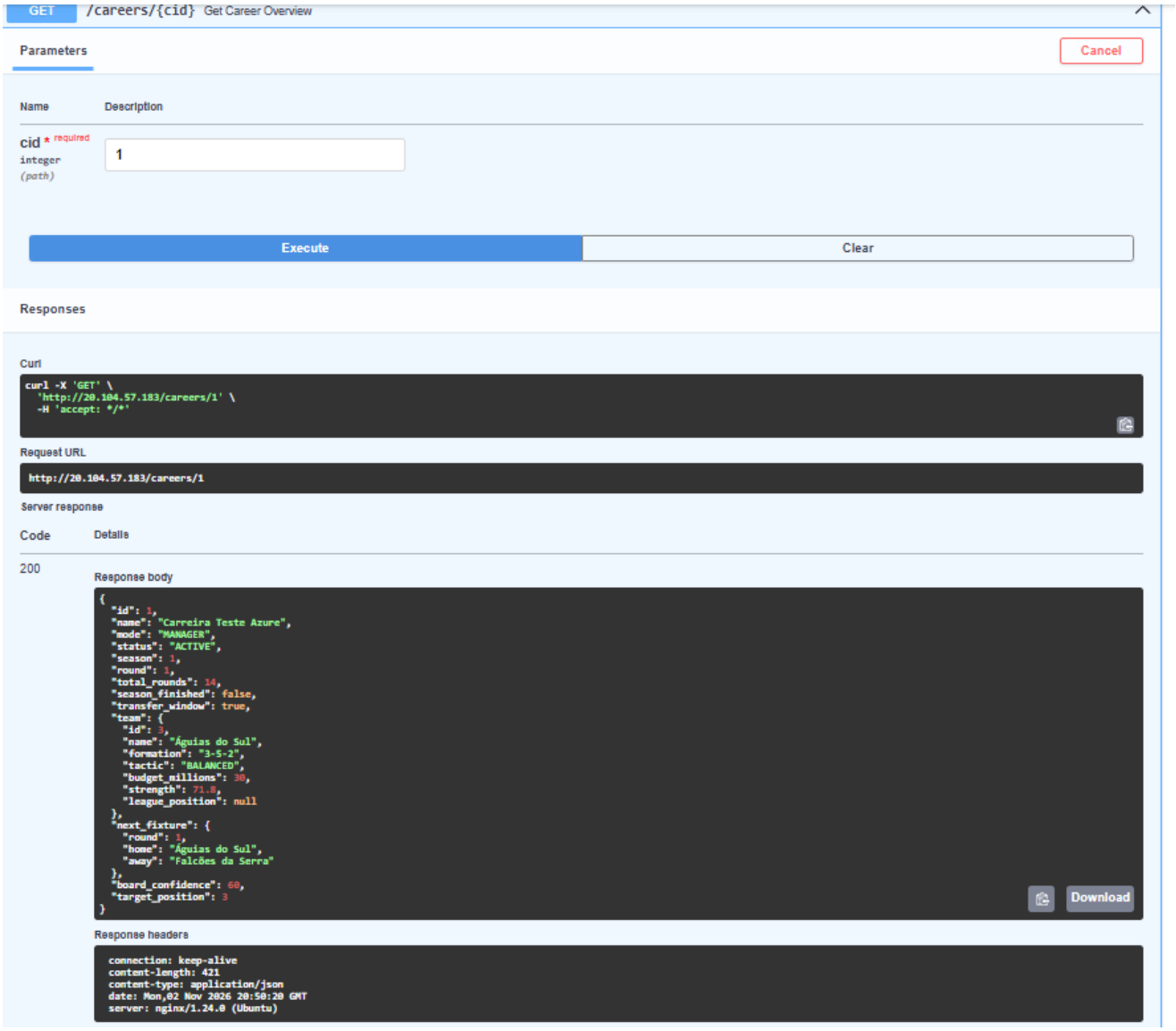


Figura 9 — Resumo da carreira por API

GET /careers/1 retorna equipe, formação, orçamento e próximo jogo.

Evidências do perfil e das propostas

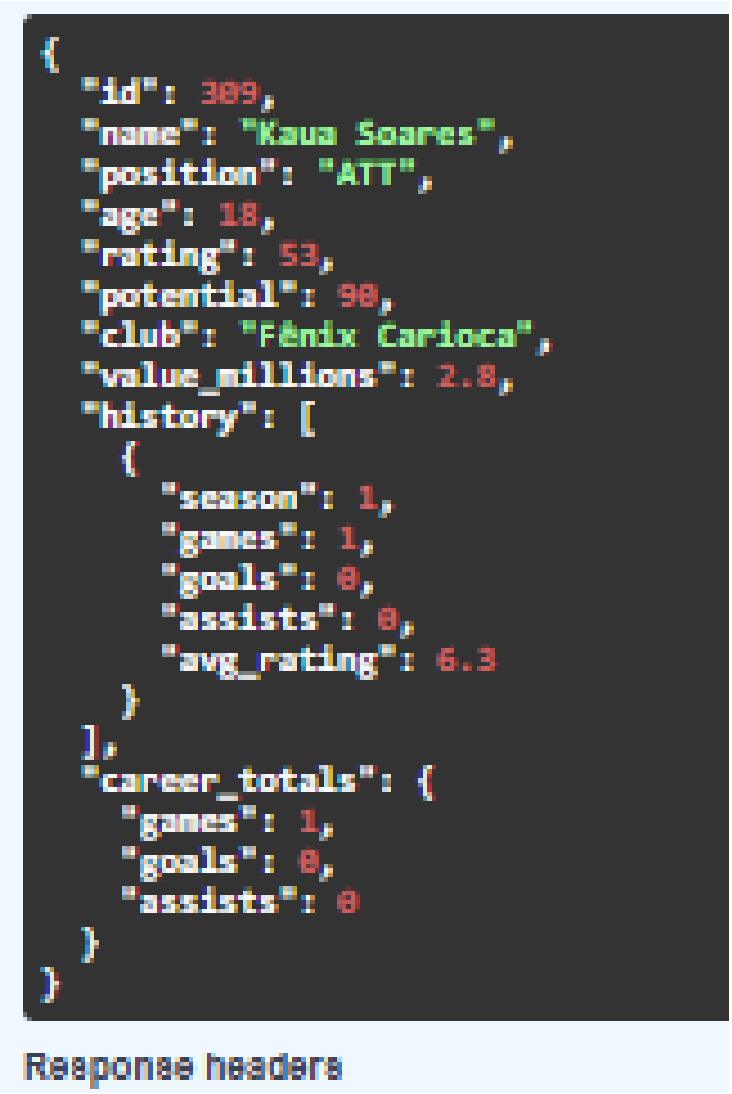


Figura 10 — Perfil e estatísticas do atleta

Histórico contém jogos e média de notas; totais de carreira estão presentes.

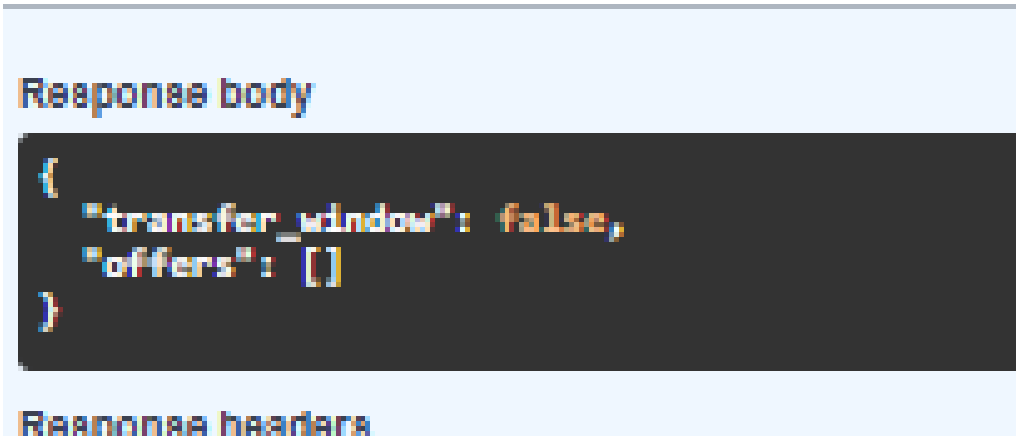


Figura 11 — Janela fechada sem propostas

transfer_window false e offers vazio formam um estado válido.

Evidências do banco e do backup

```
azureuser@vm-football-career:/opt/football-career-api$ ls -lh data/
total 48K
-rw-r--r-- 1 azureuser azureuser 48K Nov  2 20:23 career.db
azureuser@vm-football-career:/opt/football-career-api$ sqlite3 data/career.db ".tables"
careers       goals         players       teams
fixtures      offers        season_stats
azureuser@vm-football-career:/opt/football-career-api$ sqlite3 data/career.db "PRAGMA integrity_check;
"
ok
azureuser@vm-football-career:/opt/football-career-api$
```

Figura 12 — Banco SQLite e integridade

Arquivo career.db, sete tabelas e integrity_check com resultado ok.

```
(.venv) azureuser@vm-football-career:/opt/football-career-api$ python3 backup_db.py
Backup criado e validado: /opt/football-career-api/backups/career-20261102T212136Z.db
(.venv) azureuser@vm-football-career:/opt/football-career-api$ ls -lh backups/
total 76K
-rw-r--r-- 1 azureuser azureuser 76K Nov  2 21:21 career-20261102T212136Z.db
```

Figura 13 — Backup criado e validado

Arquivo de backup datado listado no diretório backups.

```
(.venv) azureuser@vm-football-career:/opt/football-career-api$ mkdir -p /tmp/football-career-restore
(.venv) azureuser@vm-football-career:/opt/football-career-api$ cp backups/career-20261102T212136Z.db \
/tmp/football-career-restore/career-restored.db
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sqlite3 /tmp/football-career-restore/career-restored.db \
"PRAGMA integrity_check;"
ok
(.venv) azureuser@vm-football-career:/opt/football-career-api$ sqlite3 /tmp/football-career-restore/career-restored.db \
"SELECT id, name, mode, status FROM careers;"
1|Carreira Teste Azure|MANAGER|ACTIVE
2|Carreira Jogador Azure|PLAYER|ACTIVE
```

Figura 14 — Consulta da cópia restaurada

integrity_check ok e carreiras MANAGER e PLAYER na cópia em /tmp.

Evidência da interface inicial

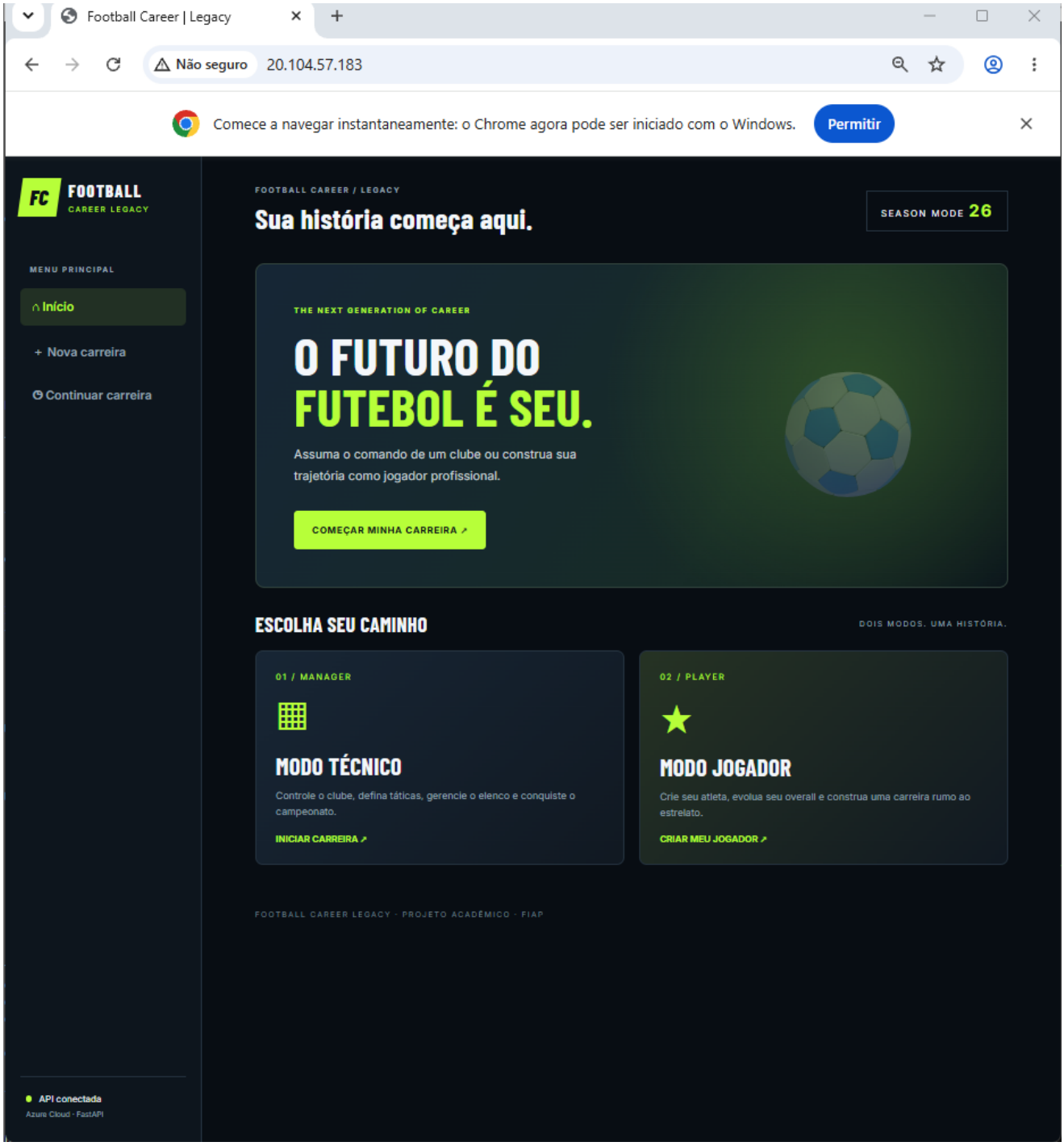


Figura 15 — Página inicial do Legacy

Frontend acessível pelo IP público, com indicação API conectada.

Evidências do modo técnico

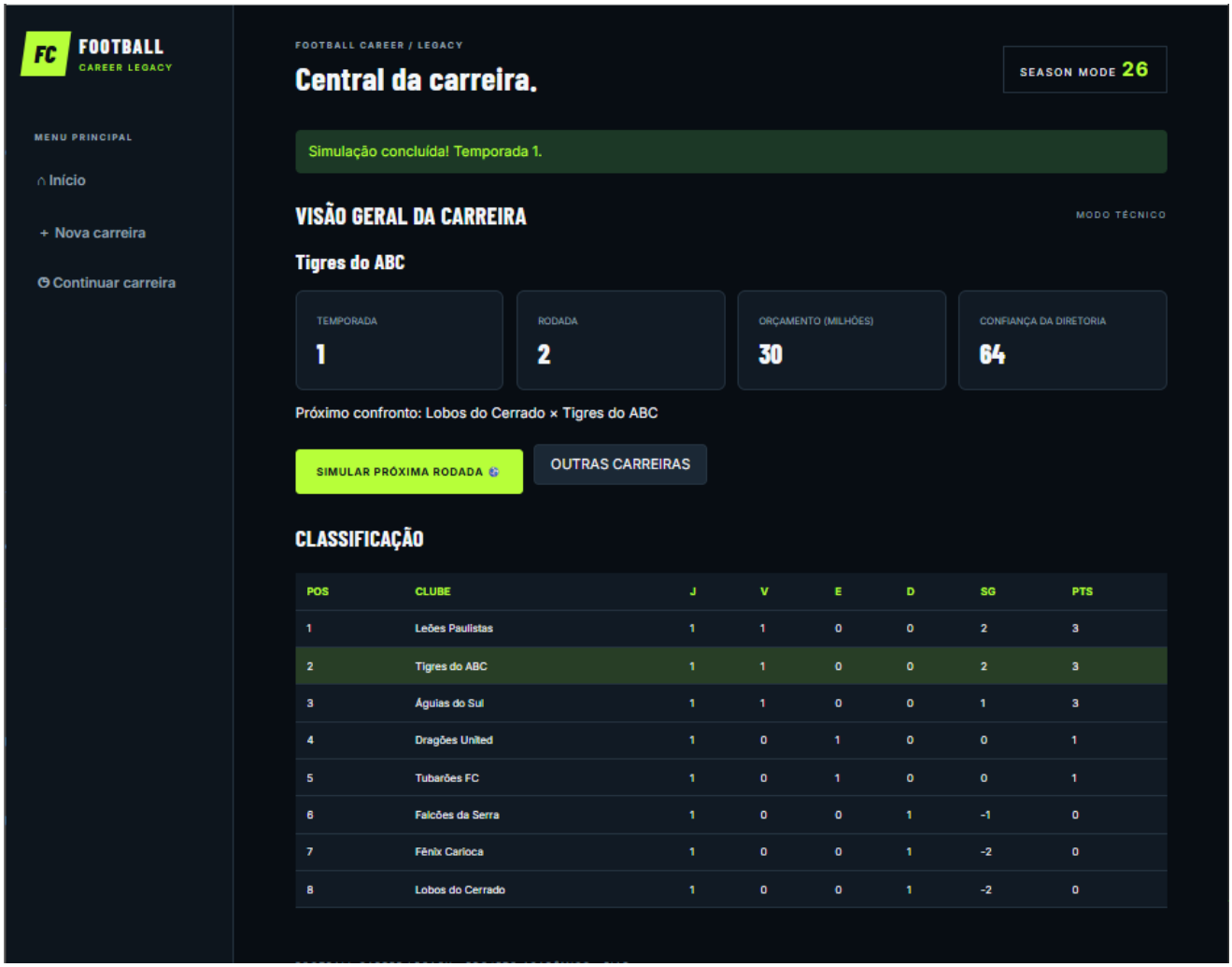


Figura 16 — Simulação pela interface

Rodada avança para 2 e confiança sobe para 64 após vitória.

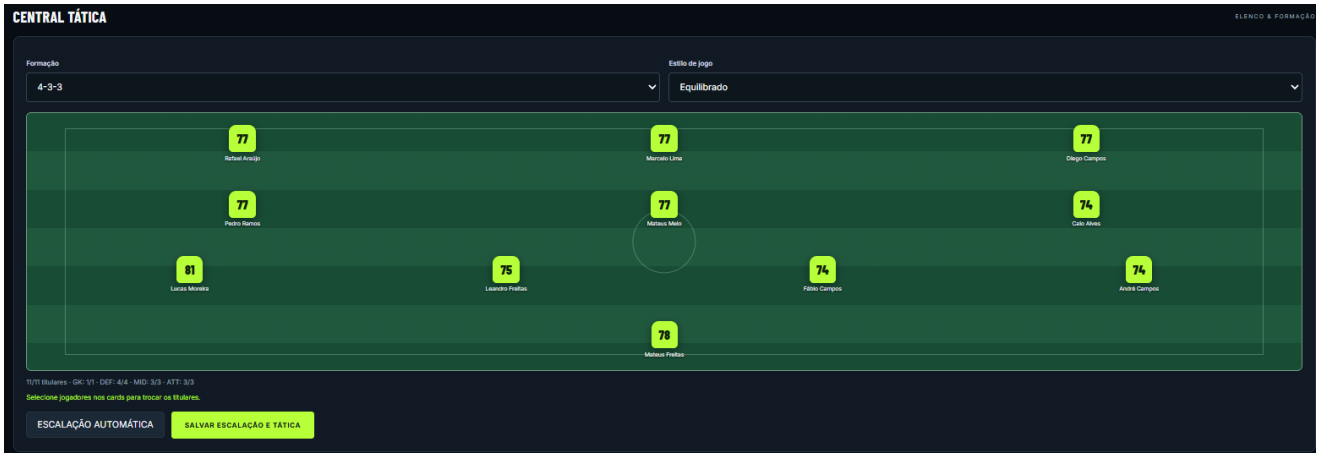


Figura 17 — Central tática com formação 4 3 3

A distribuição visual mostra 1 GK, 4 DEF, 3 MID e 3 ATT; a tela não prova reabertura após salvar.

Evidência do modo jogador

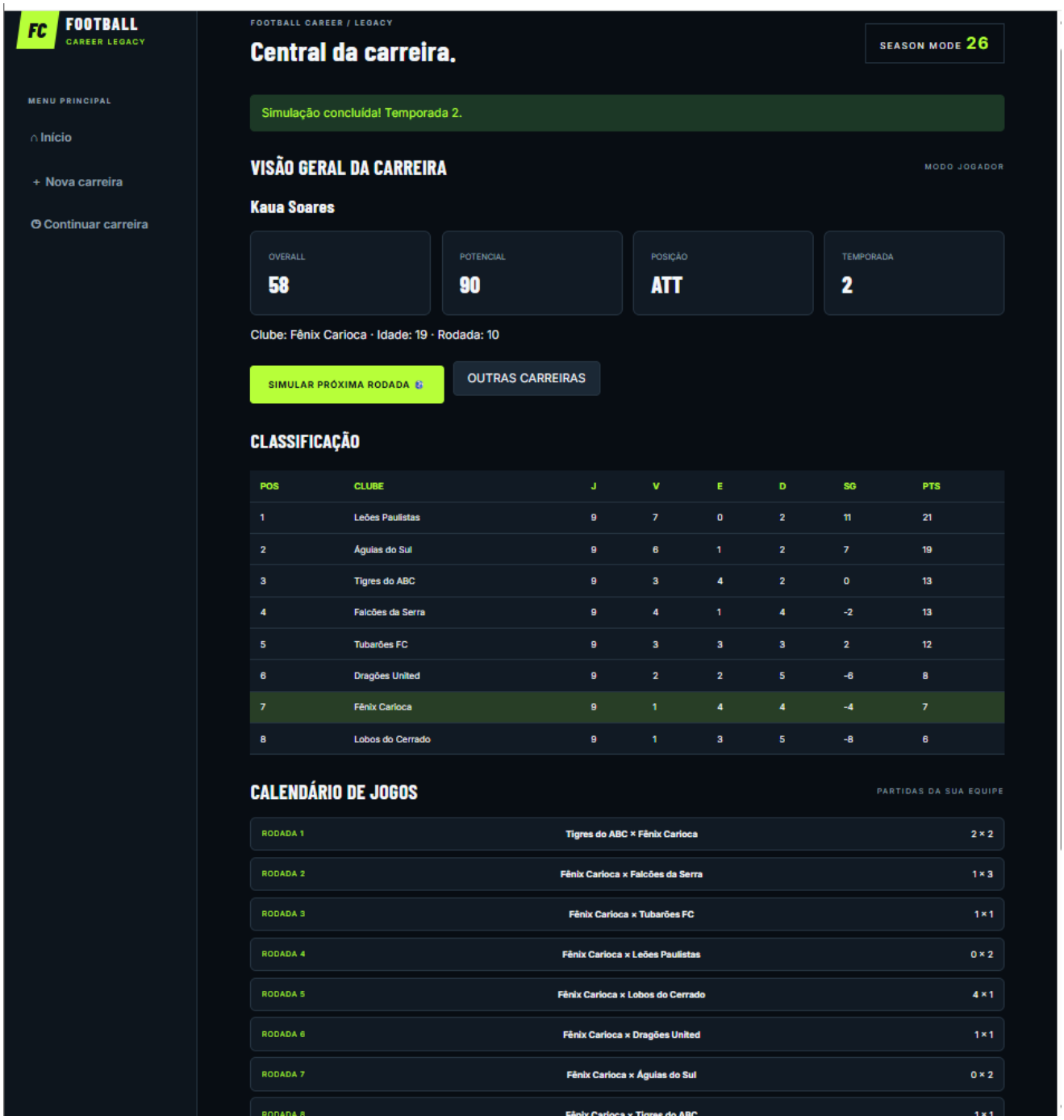


Figura 18 — Rodada 10 do modo jogador

Mensagem de simulação concluída e tabela com nove jogos disputados.